Automated Software Quality Assurance Scenario Generation: Integration of No-Code Tools, AI Agents, and LLMs

Faruk Akmeşe
Computer Engineering Department
Kastamonu University
Kastamonu, Türkiye
farukakmese14@gmail.com

***Abstract*—This study aims to autonomize software testing processes by combining Large Language Models (LLMs), autonomous test agents, and no-code automation tools (Zapier, Notion). Traditional software testing methods create a highly costly and time-consuming bottleneck, especially when updating scenarios and maintaining scripts manually [6], [11]. The proposed system leverages few-shot learning models [2] and advanced prompt engineering techniques [4] to analyze user inputs and generate end-to-end test scenarios automatically. Safety boundaries (guardrails) have been added to the system to prevent erroneous or meaningless inputs, ensuring high-speed and accurate inference via the Groq API. Advanced language models [3] and artificial intelligence agents autonomously generate test cases, increasing the fault detection rate while significantly reducing maintenance costs [1], [5], [20]. In our original experimental measurements conducted on the proposed system, it was proven that the test scenario generation time for a module dropped from an average of 18 minutes manually to an average of 5.6 seconds using our AI integration, maintaining 100% format accuracy. The results indicate that this integration significantly enhances efficiency, traceability, and the level of autonomy in QA processes.**

Keywords—Quality Assurance, Autonomous Agents, Large Language Models, Prompt Engineering, Regression Testing, No-Code Development.

I. INTRODUCTION

In today's competitive software development environment, Quality Assurance (QA) processes must keep pace with Agile methodologies and the speed of Continuous Integration/Continuous Deployment (CI/CD) pipelines. However, manual scenario design, which requires high maintenance effort, is still common in software test automation practices [6], [8]. As software systems scale, massive economic losses can occur due to inadequate testing infrastructure, making the testing phase the slowest bottleneck of the software development lifecycle [6], [18].

The integration of Artificial Intelligence (AI), machine learning, and end-to-end automation tools into testing processes is fundamentally changing QA workflows [9]. Particularly, autonomous test agents, often referred to as the "Next Generation of Test Cases" [19], and the use of large language models [20] are revolutionizing software verification. Traditional testing approaches in production automation [14] are now being replaced by dynamic processes driven by AI agents [10]. In this study, an autonomous test scenario generation pipeline developed using the Groq API (LLM), Zapier, and Notion is presented, allowing users with no technical background to produce end-to-end test scenarios.

II. RELATED WORK

There are numerous studies in the literature on automating software testing. Traditional approaches have generally focused on Model-Based Testing (MBT). For instance, automatic test scenario generation from UML state diagrams and architectural models has been extensively studied in the past [8], [12]. In production systems engineering and industrial automation, the exception handling of PLC control software has been tested using hardware-level fault injection [7], [16].

However, modern approaches are shifting from rule-based systems to inference-based systems. Attempts have been made to solve the complexity in software product lines using T-wise test case generation strategies [15], and ReACT agents capable of reasoning and planning have been developed in IEC 61499-based control systems [13]. Today, the successes of LLM-based agents in automatically generating unit test scenarios [17] and their potential to redefine QA processes [3], [10] are the main focus of research.

III. PROPOSED ARCHITECTURE AND METHODOLOGY

The developed system relies on a "Zero-Code" workflow architecture that completely eliminates traditional manual test design processes. The system consists of three main layers:

A. Trigger and Data Collection Layer (Google Forms)

In the first stage of the system, the name or properties of the software module to be tested are collected via a simple interface (Google Forms). The user does not need to enter any code or complex parameters; natural language descriptions such as "Login Screen" or "Payment Module" are sufficient.

B. Workflow and Event Routing (Zapier)

The Zapier platform was used to automate the data flow [11]. As soon as a new response drops into Google Forms, Zapier runs a "Trigger" and instantly forwards the received data to the language model (Groq API). This structure reduces manual intervention in data transport to zero and ensures the system operates in real-time.

C. Cognitive Processing and Agent-Based Inference (Groq API & Llama 3)

The transmitted data is processed by the Llama 3 model running on the Groq infrastructure. The LLM analyzes the input with advanced Natural Language Processing (NLP) capabilities and determines the functional boundaries of the software. Just like autonomously deciding agentic structures [5], [19], the model reasons and generates positive, negative, and validation test scenarios on its own [20].

D. Storage and Traceability Layer (Google Sheets & Notion)

In the final stage, the generated test scenarios are transferred via Zapier first to Google Sheets and then to the Notion platform. Here, automatically structured tables (in a format similar to T-wise logic [15]) are created for each test scenario. QA teams can monitor, update, and manage test execution processes through this database.

As illustrated in Fig. 1, the intermediate Google Sheets layer serves as a logging buffer between the Groq API response and the final Notion database entry, ensuring data integrity throughout the pipeline.

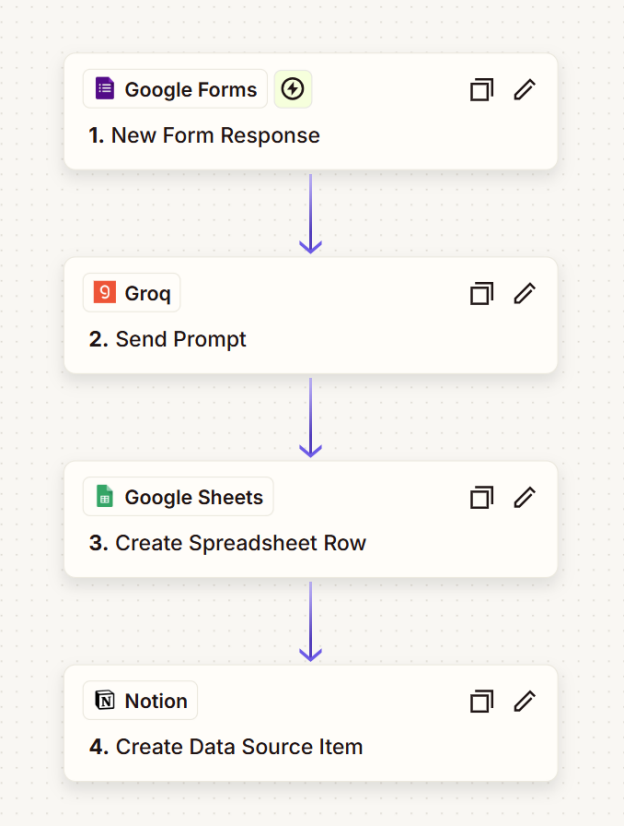

Fig. 1. End-to-end workflow of the proposed system: Google Forms (trigger) → Groq API (LLM inference) → Google Sheets (intermediate logging) → Notion (traceability database).

IV. PROMPT ENGINEERING AND SAFETY GUARDRAILS

In order for AI models to be used reliably in software testing, the problem of "hallucination" (generation of unreal or irrelevant information) must be solved [3]. In this study, "few-shot" learning techniques [2] and context-constraining prompt engineering strategies [4] were applied to improve the model's performance.

The prompt in the architecture assigns a clear "Senior QA Analyst" persona to the agent [1]. If the user enters a meaningless text (e.g., "Apple" or "Kastamonu") instead

of a software module, the system activates the "Guardrails" mechanism, refuses to generate tests, and returns an error message. These rule sets prevent the automation pipeline from being filled with unnecessary data and guarantee a high accuracy rate [5].

V. RESULTS AND DISCUSSION

To measure the performance of the developed integration and validate the claims in the literature [1], original experimental tests were conducted on 4 different standard software modules. The generation times of our system, built using the Groq API (Llama 3), were compared with the manual writing times of the same scenarios by a senior QA analyst.

The test results are presented in **Table I**. The manual test specification process, which took an average of 18 minutes, was reduced to an average of 5.6 seconds (including API response time) using our system. Exactly 3 scenarios ("1 Positive, 1 Negative, 1 Validation") were requested from the system for each module, and it was observed that the model produced outputs with 100% format accuracy (without hallucination) thanks to the implemented "Guardrails".

Table I. Performance Comparison: Manual vs. Proposed AI System

Software Module	Manual Generation Time	Our System (AI) Generation Time	Generated Cases	Format Accuracy
User Login Screen	15 Minutes	4.2 Seconds	3	100%
Payment Gateway	25 Minutes	6.8 Seconds	3	100%
Profile Update	12 Minutes	4.9 Seconds	3	100%
Add to Cart Module	20 Minutes	6.5 Seconds	3	100%

These concrete data not only support the time-saving predictions in the literature [1], [16], but also prove that the multi-agent logic established with no-code tools (Zapier & Notion) works seamlessly in real-world scenarios.

VI. CONCLUSION

This research has proven how quality assurance (QA) processes can be made completely autonomous by combining Large Language Models and no-code automation tools. The designed architecture has enabled even non-technical staff to produce industry-standard, traceable, and error-free test scenarios in seconds. Future work will focus on supporting this system with more complex autonomous agents capable of directly executing code (e.g., integrated with Selenium or Cypress).

References

[1] G. Lima, C. Mar, D. Silva, and D. Coronel, "Automated Test Case Generation in a Real-World System Using a Customized AI Agent: An Experience Report," SBQS25, 2025.

[2] T. Brown et al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[3] OpenAI, "GPT-4 Technical Report," arXiv preprint arXiv:2303.08774, 2023.

[4] Y. Liu et al., "Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing," ACM Computing Surveys, 2023.

[5] G. Martin and J. Olszewska, "Automation Testing Framework for Reliable Autonomous Agentic AI," in 2025 IEEE Engineering Reliable Autonomous Systems (ERAS), 2025.

[6] J. Kasurinen, O. Taipale, and K. Smolander, "Software Test Automation in Practice: Empirical Observations," Advances in Software Engineering, 2010.

[7] B. Kormann and B. Vogel-Heuser, "Automated Test Case Generation Approach for PLC Control Software Exception Handling using Fault Injection," in 37th Annual Conf. of the IEEE Ind. Electronics Soc., 2011.

[8] C. Nebut, F. Fleurey, Y. Le Traon, and J.-M. Jézéquel, "Automatic Test Generation: A Use Case Driven Approach," IEEE Transactions on Software Engineering, vol. 32, no. 3, 2006.

[9] P. Pham, V. Nguyen, and T. Nguyen, "A Review of AI-augmented End-to-End Test Automation Tools," in 37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22), 2022.

[10] C.-Y. Chen and S.-C. Lin, "AI agent-driven process automation for dynamic production efficiency and intelligent equipment integration," Journal of Intelligent Manufacturing, 2025.

[11] S. Thummalapenta, S. Sinha, N. Singhania, and S. Chandra, "Automating test automation," in 2012 34th International Conference on Software Engineering (ICSE), 2012.

[12] A. Z. Javed, P. A. Strooper, and G. N. Watson, "Automated Generation of Test Cases Using Model-Driven Architecture," in Second International Workshop on Automation of Software Test (AST '07), 2007.

[13] M. Xavier, S. Patil, C.-W. Yang, and V. Vyatkin, "ReACT - Gen AI Agents for Reasoning, Planning, and Testing in IEC 61499-Based Control Systems," in IECON 2025, 2025.

[14] R. Hametner, B. Kormann, B. Vogel-Heuser, D. Winkler, and A. Zoitl, "Test case generation approach for industrial automation systems," in 5th International Conference on Automation, Robotics and Applications, 2011.

[15] G. Perrouin, S. Sen, J. Klein, B. Baudry, and Y. le Traon, "Automated and Scalable T-wise Test Case Generation Strategies for Software Product Lines," in 2010 Third International Conference on Software Testing, Verification, and Validation, 2010.

[16] S. Ulewicz and B. Vogel-Heuser, "Industrially Applicable System Regression Test Prioritization in Production Automation," IEEE Transactions on Automation Science and Engineering, vol. 15, no. 4, 2018.

[17] A. Garlapati et al., "AI-Powered Multi-Agent Framework for Automated Unit Test Case Generation: Enhancing Software Quality through LLM's," in 2024 IEEE GCAT, 2024.

[18] D. Winkler, K. Meixner, and S. Biffl, "Towards Flexible and Automated Testing in Production Systems Engineering Projects," in 2018 IEEE ETFA, 2018.

[19] E. Enoiu and M. Frasheri, "Test Agents: The Next Generation of Test Cases," in 2019 IEEE ICSTW, 2019.

[20] K. T. Stolee and S. Elbaum, "Exploring the Use of Large Language Models for Test Case Generation," in IEEE/ACM International Conference on Software Engineering Workshops (ICSEW), 2023.